

CONTENTS

- Overview
- 1 The Neuron as a Function: Vector In, Signal Out
- 2 Turning Real-World Data Into a Vector
- 3 Weights: Encoding Learned Importance
- 4 How Neurons in the Same Layer Specialize
- 5 Squashing the Weighted Sum: Activation and Bias
- 6 Why Nonlinearity Lets Networks Learn Complex Patterns
- 7 From One Neuron to a Network: Stacking, Weights, and Training
- Glossary
- Check yourself
- Where to go next

Inside a Neuron: The Building Blocks of a Neural Network & AI

Compute a single artificial neuron's output by hand and understand what weights, bias, and activation functions actually do.

For people who have heard the term 'neural network' and want to understand, in concrete arithmetic, what a single neuron computes · 27 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- Comfort with basic algebra: multiplication, addition, and summing a list of numbers
- A rough sense of what a neural network is used for, e.g. taking data in and producing a prediction

Overview

A neural network looks intimidating from the outside, but it is built from one small, repeated unit: the artificial neuron. Each neuron takes in a list of numbers, multiplies them by a set of learned importance values, adds them up, and squashes the result into a signal. Nothing more mysterious happens at the level of one neuron than a dot product, an addition, and a squashing function.

This lesson walks through that computation step by step, using a running example: a neuron that helps predict a house's price from features like square footage and bedroom count. You'll see how raw data becomes a vector, how weights encode which features a neuron cares about, why a bias term matters, and why the final squashing step (the activation function) is what gives neural networks the power to learn anything more complex than a straight line.

By the end, you should be able to take a small set of inputs, weights, and a bias, and compute a neuron's output yourself with a calculator — which is the best way to demystify what training a neural network actually adjusts.

KEY TAKEAWAYS

- ✓ A neuron turns an input vector into a single output signal by computing a weighted sum of the inputs, adding a bias, then passing the result through an activation function.
- ✓ Weights encode how much a specific neuron cares about each input feature; every neuron in a layer sees the same input vector but specializes by learning a different set of weights.
- ✓ The bias shifts a neuron's activation threshold, letting it fire even when the weighted sum of its inputs is small or zero.
- ✓ Activation functions like sigmoid or ReLU introduce nonlinearity. Without them, stacking any number of neurons would still compute nothing more than one overall linear function.
- ✓ A neuron's activation level (its output after the activation function) is a measure of how strongly the pattern that neuron detects is present in the current input.
- ✓ Training a network means repeatedly adjusting every neuron's weights and biases to reduce prediction error; backpropagation is the algorithm that works out how to adjust them, though it isn't covered in depth here.
- ✓ Tools like MLflow don't change what a neuron computes — they log and visualize how weights and biases evolve across training runs so engineers can compare configurations.

01 The Neuron as a Function: Vector In, Signal Out

0:00

A neural network is a collection of neurons, and each neuron is just a small function: it takes a vector of numbers in and produces one output signal.

Strip away the diagrams and a neural network is a large number of identical small units wired together. Each of those units is an artificial neuron, and understanding one neuron is most of the work of understanding the whole system, because every neuron performs the same three-step computation on whatever numbers arrive at its input.

A neuron's input is a vector: an ordered list of numbers where each position represents one measurable property of the data point being processed. Its output is a single number, called a signal or activation, that gets passed on to the next layer of neurons (or, if it's the last layer, becomes part of the network's final prediction). Everything a neuron does — deciding which features matter, combining them, and deciding whether to 'fire' — happens inside that one input-vector-to-output-number transformation.

KEY POINTS

- A neural network is a collection of neurons; each neuron does an identical three-step computation.
- A neuron's input is always a vector (an ordered list of numbers); its output is always a single number.
- That single output number is called a signal or activation, and it feeds into the next layer.
- Understanding one neuron's math is the foundation for understanding the whole network.

One neuron, one function

Think of a neuron as a small function `neuron(x)` that accepts a vector $x = [x_1, x_2, x_3, x_4]$ and returns one number. Everything in this lesson is about what happens inside that function.

```
def neuron(x, weights, bias):  
    # x, weights are same-length vectors; bias is a single number  
    # returns one number: the neuron's output signal  
    ...
```

02 Turning Real-World Data Into a Vector

0:40

Before a neuron can process data, real-world features have to be encoded as normalized numbers and arranged into an input vector.

Consider training a network to predict housing prices. A house has features that matter to price: square footage, number of bedrooms, number of bathrooms, zip code. None of these are naturally on the same numeric scale — square footage might be in the thousands, bedroom count is a small integer, zip code is a categorical label. Before any of this can enter a neuron, it gets normalized: converted to values on a comparable scale (commonly between 0 and 1) so that no single feature dominates the computation purely because of the units it happens to be measured in.

Once normalized, the features are arranged in a fixed order to form the input vector for that house — for example `[square_footage, bedrooms, bathrooms, zip_code]` becoming something like `[0.8, 0.5, 0.3, 0.6]`. Every house in the dataset gets encoded the same way, in the same feature order, so that position 1 always means 'square footage' no matter which house's vector you're looking at. This input vector is what actually goes into every neuron in the network's first layer.

KEY POINTS

- Raw features (square footage, bedrooms, bathrooms, zip code) are normalized to a common numeric scale before use.
- Normalization prevents a feature with naturally large raw values from dominating the computation just because of its units.
- The normalized features are arranged in a fixed, consistent order to form the input vector.
- Every data point (every house) is encoded with the same feature order, so the same vector position always means the same feature.

Encoding one house as a vector

A house with large square footage, an average number of bedrooms, few bathrooms, and a mid-range zip code might normalize to the vector below. This exact vector is what gets fed into every neuron in the first layer.

feature	raw value	normalized
sq_footage	2,400	0.8
bedrooms	3	0.5
bathrooms	1	0.3
zip_code	94103	0.6

```
input_vector = [0.8, 0.5, 0.3, 0.6]
```

03 Weights: Encoding Learned Importance

1:25

Each input feature is multiplied by a weight — a learned number representing how important that feature is to this particular neuron — and the products are summed.

A neuron doesn't treat every input feature equally. Each feature in the input vector is paired with a corresponding weight: a number that represents how much that feature matters to this neuron's decision. If training determines that square footage strongly predicts price, the weight attached to square footage grows large (e.g. 0.9). If bathroom count turns out to matter less, its weight stays small (e.g. 0.2).

To combine the input vector with the weights, the neuron multiplies each feature by its matching weight, then adds all of those products together. This is called the weighted sum. For an input vector $x = [x_1, x_2, x_3, x_4]$ and weights $w = [w_1, w_2, w_3, w_4]$, the weighted sum is $x_1*w_1 + x_2*w_2 + x_3*w_3 + x_4*w_4$.

Mathematically this is a dot product between the input vector and the weight vector — the weighted sum is nothing more than 'how well does this input line up with what this neuron is looking for.'

KEY POINTS

- Every input feature has a matching weight: a learned number representing that feature's importance to this specific neuron.
- A large weight means the neuron pays close attention to that feature; a small weight means it mostly ignores it.
- The weighted sum is computed by multiplying each feature by its weight and adding the results together.
- The weighted sum is mathematically a dot product between the input vector and the weight vector.

Computing the weighted sum

Using the housing input vector from the previous section and a set of weights where this neuron cares most about square footage (0.9) and least about bathrooms (0.2), the weighted sum is the sum of each feature times its weight.

```
x = [0.8, 0.5, 0.3, 0.6] # sqft, bedrooms, bathrooms, zip
w = [0.9, 0.4, 0.2, 0.5] # this neuron's learned weights

weighted_sum = (x[0]*w[0]) + (x[1]*w[1]) + (x[2]*w[2]) + (x[3]*w[3])
#           = (0.8*0.9) + (0.5*0.4) + (0.3*0.2) + (0.6*0.5)
#           =  0.72   +  0.20   +  0.06   +  0.30
#           =  1.28
```

04 How Neurons in the Same Layer Specialize

2:09

Every neuron in a layer receives the identical input vector, but each has its own independent weights, so different neurons learn to detect different patterns.

A layer of a neural network contains many neurons, and every one of them sees the exact same input vector — the same four housing features in this example. What differs between neurons is not the data they receive, but their weights. One neuron might develop large weights for square footage and zip code, effectively specializing in detecting 'large homes in expensive areas.' A neighboring neuron might instead develop large weights for bedroom and bathroom counts, specializing in a different pattern entirely.

This specialization isn't designed by hand — it emerges from training. If a particular neuron's weighted sum consistently helps the network reduce its prediction error when a certain pattern is present (say, large suburban homes), the training process reinforces the weights that produced that useful signal, and the

neuron becomes more strongly tuned to that pattern over time. Repeat this across many neurons and many training examples, and each neuron in a layer ends up responsible for detecting a different useful signal in the data, without ever being explicitly told what to look for.

KEY POINTS

- All neurons in a layer receive the same input vector; only their weights differ.
- Different weight sets cause different neurons to respond strongly to different feature combinations.
- Specialization emerges from training: weights that help reduce prediction error get reinforced.
- Across many neurons, the layer collectively covers many different useful patterns in the data.

Two neurons, same input, different focus

Neuron A's weights emphasize square footage and zip code; Neuron B's weights emphasize bedrooms and bathrooms. Given the identical input vector, they produce different weighted sums because they're each looking for a different pattern.

```
x = [0.8, 0.5, 0.3, 0.6]      # same input for both neurons

w_A = [0.9, 0.1, 0.1, 0.7]    # Neuron A: cares about sqft + zip
w_B = [0.1, 0.8, 0.7, 0.1]    # Neuron B: cares about bed + bath

sum_A = sum(xi*wi for xi, wi in zip(x, w_A)) # = 0.72+0.05+0.03+0.42 = 1.22
sum_B = sum(xi*wi for xi, wi in zip(x, w_B)) # = 0.08+0.40+0.21+0.06 = 0.75
```

05 Squashing the Weighted Sum: Activation and Bias

2:52

The raw weighted sum can be any number, so a neuron adds a bias and then applies an activation function like sigmoid to squash the result into a bounded, usable range.

A weighted sum by itself is unbounded — depending on the inputs and weights, it could come out as -50 or 500, which is not a convenient signal to pass along or compare between neurons. To fix this, a neuron applies an activation function to the weighted sum. The sigmoid function compresses any real number into a range between 0 and 1: very negative inputs get pushed toward 0, very positive inputs get pushed toward 1, and inputs near zero land near 0.5. This makes it easy for the rest of the network to treat the output as something like a confidence: how strongly is this neuron's pattern present?

Before applying the activation function, the neuron adds one more term to the weighted sum: the bias, usually written as b . The bias is a single learned number, independent of any input feature, that shifts the threshold at which the neuron activates. Without a bias, a neuron could only fire when its weighted sum is

naturally positive; the bias lets a neuron fire even when all its inputs are small, or conversely, makes it harder to fire even with sizeable inputs, depending on the sign it learns.

Putting it together, the full computation for one neuron is: $\text{activation} = \text{sigmoid}(\text{weighted_sum} + \text{bias})$, or written out, $\text{sigmoid}(x_1*w_1 + x_2*w_2 + x_3*w_3 + x_4*w_4 + b)$. Modern networks mostly use ReLU instead of sigmoid for efficiency — sigmoid needs an exponential calculation for every neuron on every forward pass, while ReLU only needs a comparison against zero — but sigmoid is easier to reason about conceptually, which is why it's used here.

KEY POINTS

- A raw weighted sum is unbounded, which makes it hard to interpret or compare across neurons.
- The sigmoid activation function squashes any input into the range (0, 1).
- The bias is a single learned number added before activation; it shifts the neuron's activation threshold independent of the inputs.
- The full neuron computation is $\text{sigmoid}(\text{weighted_sum} + \text{bias})$.
- Most modern networks use ReLU instead of sigmoid for computational efficiency, though sigmoid is conceptually simpler.

Full worked example: one neuron's output by hand

Take the weighted sum computed earlier (1.28), add a bias of -0.5, then apply the sigmoid function. This is the complete, concrete computation a single neuron performs for one house.

```
import math

x = [0.8, 0.5, 0.3, 0.6]  # normalized: sqft, bedrooms, bathrooms, zip
w = [0.9, 0.4, 0.2, 0.5]  # this neuron's learned weights
b = -0.5                  # this neuron's learned bias

# Step 1: weighted sum
weighted_sum = sum(xi * wi for xi, wi in zip(x, w))
# weighted_sum = 0.72 + 0.20 + 0.06 + 0.30 = 1.28

# Step 2: add bias
z = weighted_sum + b
# z = 1.28 - 0.5 = 0.78

# Step 3: squash with sigmoid
activation = 1 / (1 + math.exp(-z))
# activation ≈ 0.686

print(round(activation, 3))  # 0.686
# Interpretation: this neuron is fairly strongly activated (0.686 is
# well above the 0.5 midpoint), meaning the pattern it detects is
# clearly present in this house's features.
```

06 Why Nonlinearity Lets Networks Learn Complex Patterns

3:35

The activation function is what makes a network nonlinear; without it, stacking any number of neurons would collapse into a single linear model.

The weighted sum step (multiply, then add) is a purely linear operation. If a network only ever computed weighted sums and passed them directly to the next layer with no activation function in between, then no matter how many layers you stacked, the entire network would be mathematically equivalent to one single linear function. Layers of pure weighted sums compose into another weighted sum — you gain more parameters, but not more expressive power.

The activation function breaks that collapse. Because sigmoid (or ReLU) is a nonlinear function, feeding its output into another layer's weighted sum produces something that can no longer be reduced to a single straight-line relationship between input and output. This is what allows a multi-layer network to represent curved decision boundaries, combinations of features, and other complex patterns that a linear model fundamentally cannot capture, no matter how it's tuned.

The activation function's output — the activation level — has a direct interpretation: a value close to 1 (for sigmoid) means the neuron's weighted sum was strongly positive after the bias was applied, i.e. the pattern this neuron detects is strongly present in the current input. A value close to 0 means that pattern is essentially absent. This is the sense in which an activation function 'decides whether a neuron fires.'

KEY POINTS

- Stacking layers of pure weighted sums, with no activation function, still only computes one overall linear function.
- The activation function's nonlinearity is what lets multi-layer networks represent curved, complex relationships.
- An activation level near 1 (sigmoid) signals the neuron's pattern is strongly present; near 0 signals it's absent.
- This is why activation functions are sometimes described as deciding 'whether a neuron fires.'

Two linear layers collapse into one

Without an activation function between them, a two-layer network is algebraically identical to a single layer. Here, layer 1 computes $h = 2x + 1$ and layer 2 computes $y = 3h - 4$; substituting shows the whole thing reduces to one linear function of x .

```
# Layer 1 (no activation):  $h = 2x + 1$ 
# Layer 2 (no activation):  $y = 3h - 4$ 

# Substitute h into layer 2:
#  $y = 3(2x + 1) - 4$ 
#  $y = 6x + 3 - 4$ 
#  $y = 6x - 1$ 

# This is exactly the form of a single linear layer:  $y = Wx + B$ 
# with  $W = 6$  and  $B = -1$ . The extra layer added no new capability.
```

07 From One Neuron to a Network: Stacking, Weights, and Training

4:20

A single neuron can only capture one relationship, but stacking many, fully connected between layers, lets a network learn combinations of patterns; training is the process of tuning every weight and bias.

A single neuron, however well-tuned, can only capture one relationship between its inputs and an output signal. Real predictive power comes from stacking many neurons: in a typical fully connected network, every neuron in one layer connects to every neuron in the next layer, each connection carrying its own

weight, and each neuron carrying its own bias. This gives the network a large number of independent 'dials' — every weight and every bias — that can be individually tuned.

Training a neural network is the process of searching for the configuration of all those dials that makes the network's predictions as accurate as possible. Concretely, during training the network tweaks weights and biases to reduce prediction error; the algorithm responsible for figuring out which direction to nudge each dial is called backpropagation, which works backward from the network's error to assign a share of the blame to every weight and bias in the network. Backpropagation is a substantial topic on its own and isn't covered in depth here, but the key idea to take away is that 'training' has a precise meaning at the level of a single neuron: it means adjusting that neuron's weights and bias.

Because a real network can have millions of these dials, engineers use tooling to keep training manageable. Open-source projects like MLflow let engineers log and compare training runs, visualizing how a model's weights and biases change over time and how different configurations affect overall performance. MLflow doesn't change what a neuron computes — it's an observability layer on top of the same weighted-sum-plus-bias-plus-activation computation this lesson has walked through.

KEY POINTS

- A single neuron can only capture one relationship; networks gain power by stacking many neurons.
- In a fully connected network, every neuron connects to every neuron in the next layer, each connection with its own weight.
- Training means searching for the weight and bias values across the whole network that minimize prediction error.
- Backpropagation is the algorithm that determines how to adjust every weight and bias to reduce error (a topic beyond this lesson).
- Tools like MLflow track and visualize how weights and biases evolve during training, without changing the underlying neuron computation.

Counting the tunable dials in a small layer

Even a tiny fully connected layer has many independent weights and biases. A layer of 3 neurons, each receiving the 4-feature housing vector, has $3 \times 4 = 12$ weights plus 3 biases: 15 numbers that training has to tune.

```
num_inputs = 4      # sqft, bedrooms, bathrooms, zip
num_neurons = 3      # neurons in this layer

num_weights = num_inputs * num_neurons  # 4 * 3 = 12
num_biases  = num_neurons                # 3 (one per neuron)

total_tunable_params = num_weights + num_biases  # 15
```

Glossary

Neuron

The basic unit of a neural network. It takes a vector of numbers as input and produces one output number by computing a weighted sum, adding a bias, and applying an activation function.

Vector

An ordered list of numbers, where each position represents a specific measurable feature of the data being described.

Weight

A learned number attached to one input feature of one neuron, representing how much that feature matters to that neuron's output.

Weighted sum

The result of multiplying each input by its matching weight and adding all the products together; mathematically a dot product of the input vector and the weight vector.

Bias

A single learned number added to a neuron's weighted sum before activation. It shifts the neuron's activation threshold independent of any input feature.

Activation function

A function applied to a neuron's weighted sum (plus bias) that squashes it into a bounded, usable output and introduces nonlinearity into the network.

Sigmoid function

An activation function that compresses any real number into the range between 0 and 1, commonly used to conceptualize a neuron's output as a confidence level.

ReLU

Rectified Linear Unit, an activation function that outputs the input directly if it's positive and 0 otherwise. Used more often than sigmoid in modern networks for its computational efficiency.

Activation level

A neuron's output after its activation function has been applied; a higher activation level means the pattern that neuron detects is more strongly present in the current input.

Nonlinearity

A property of a function whose output isn't a fixed straight-line multiple of its input. Activation functions must be nonlinear for stacking multiple layers of neurons to add any representational power.

Layer

A group of neurons that all receive the same input vector, each computing its own weighted sum, bias, and activation independently.

Backpropagation

The training algorithm that works backward from a network's prediction error to determine how much each weight and bias contributed to that error, so they can be adjusted to reduce it.

Check yourself

Answer first, then open each one.

Why does a neural network need a nonlinear activation function instead of just passing the weighted sum straight to the next layer?

Because a weighted sum is a purely linear operation, and stacking any number of purely linear layers is mathematically equivalent to a single linear layer — you'd gain more tunable numbers but not more expressive power. A nonlinear activation function (sigmoid, ReLU, etc.) breaks that collapse, which is what lets a multi-layer network represent curved, complex relationships between inputs and outputs.

Two neurons in the same layer receive the exact same input vector. Why can they end up detecting completely different patterns?

Because each neuron has its own independent set of weights (and its own bias). Training adjusts each neuron's weights separately based on how much that neuron's output helps reduce the network's overall prediction error, so different neurons drift toward emphasizing different feature combinations even though the input they see never changes.

What does the bias term let a neuron do that the weights alone cannot?

The bias shifts a neuron's activation threshold independent of any input. Without it, a neuron's weighted sum would be forced toward 0 whenever the inputs themselves are small, making it impossible for the neuron to fire in that case. The bias lets a neuron fire even when all its inputs are small (or, with the opposite sign, makes it harder to fire even with sizeable inputs).

If a neuron's weighted sum plus bias comes out very negative, what does its sigmoid output tell you about that input?

Sigmoid squashes very negative values toward 0, so a very negative weighted sum produces an activation close to 0. That means the neuron is effectively 'off' for this input: the specific pattern this neuron has learned to detect is essentially absent from the current data point.

The video says most modern networks use ReLU instead of sigmoid. What's the stated reason, and what is that reason actually about?

Efficiency. Sigmoid requires computing an exponential for every neuron on every forward pass through the network, while ReLU only requires comparing a number against zero. At the scale of modern networks with millions of neurons, that difference in computation adds up, which is why ReLU is preferred in practice even though sigmoid is easier to reason about conceptually.

Where to go next

- Implement a single neuron in NumPy (dot product + bias + sigmoid) and confirm it reproduces the hand-computed 0.686 activation from this lesson.
- Plot the sigmoid and ReLU functions side by side to see visually how each squashes or thresholds its input differently.
- Read up on backpropagation and gradient descent, the mechanism mentioned but not explained here, to see exactly how a network decides which direction to nudge each weight and bias.
- Install MLflow and log a few training runs of a toy model to see how weights and biases are tracked and compared in practice.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.